

# Documentação Completa



Como criar do zero um **Portfólio Pessoal** em HTML, CSS e JavaScript puro, com Painel ADM para cadastrar projetos, criação do repositório no GitHub e hospedagem.

Tema visual: **Azul** + **Vermelho** sobre fundo Preto

# Índice

1. Visão geral do projeto
2. Estrutura de pastas e arquivos
3. Criando o repositório no GitHub (passo a passo)
4. Configurando o Git no seu computador e conectando ao repositório
5. HTML da página principal — comentado linha a linha
6. CSS do tema escuro — comentado bloco a bloco
7. JavaScript da página pública — comentado linha a linha
8. Painel ADM — Opção simples com localStorage — comentado
9. Painel ADM — Opção avançada com a API do GitHub — comentado
10. Publicando as alterações (git add, commit, push)
11. Hospedagem: GitHub Pages, Netlify e Vercel
12. Checklist final

# 1. Visão geral do projeto

O objetivo é construir um site de portfólio com duas partes:

- **Site público** ( `index.html` ): o que qualquer visitante vê — seu nome, uma seção sobre você e a lista de projetos, cada um com título, descrição e link para o repositório no GitHub.
- **Painel ADM** ( `admin.html` ): uma página separada, só para você, usada para cadastrar novos projetos.

**Ponto essencial antes de começar:** como o site vai rodar em uma hospedagem estática (GitHub Pages), ele não tem um servidor "vivo" rodando o tempo todo com banco de dados. Por isso, o Painel ADM precisa de uma estratégia específica para funcionar — isso é explicado em detalhes nas seções 8 e 9, com duas opções possíveis.

## 2. Estrutura de pastas e arquivos

Antes de escrever qualquer código, organize as pastas assim no seu computador:

```
meu-portfolio/
├──
├── index.html           // página pública do portfólio
├── admin.html           // painel de administração
├──
├── css/
│   └── style.css        // todo o estilo visual (tema escuro)
├──
├── js/
│   ├── projetos.js      // onde ficam os dados/lista de projetos
│   ├── main.js           // lógica da página pública
│   └── admin.js          // lógica do painel administrativo
├──
├── assets/
│   └── (fotos, ícones, etc.)
├──
└── projetos.json        // só será usado na Opção Avançada (seção 9)
```

Crie essa estrutura de pastas manualmente no seu sistema operacional, ou pelo terminal:

```
mkdir meu-portfolio      # cria a pasta principal
cd meu-portfolio         # entra na pasta criada
mkdir css js assets      # cria as subpastas css, js e assets
type nul > index.html     # (Windows) cria o arquivo index.html vazio
type nul > admin.html     # (Windows) cria o arquivo admin.html vazio
touch index.html admin.html # (Mac/Linux) cria os dois arquivos HTML vazios
```

### 3. Criando o repositório no GitHub

Antes de subir o código, você precisa criar o repositório dentro da sua conta do GitHub. Siga estes passos:

- 1 Acesse <https://github.com> e faça login na sua conta (crie uma gratuitamente em [github.com/signup](https://github.com/signup) caso ainda não tenha).
- 2 No canto superior direito, clique no ícone "+" e depois em **"New repository"**.
- 3 Em **"Repository name"**, digite um nome, por exemplo: `meu-portfolio`.
- 4 Em **"Description"** (opcional), escreva algo como "Meu site de portfólio pessoal".
- 5 Marque a opção **"Public"** (precisa ser público para o GitHub Pages funcionar gratuitamente e para a Opção Avançada do ADM conseguir ler o `projetos.json` sem autenticação).
- 6 Não marque para adicionar README, `.gitignore` ou licença agora — deixe tudo desmarcado, pois você vai subir seus próprios arquivos.
- 7 Clique em **"Create repository"**.
- 8 O GitHub vai mostrar uma página com comandos de exemplo (guarde essa URL, ela será usada na próxima seção). A URL do repositório será algo como:

```
https://github.com/SEU-USUARIO/meu-portfolio.git
```

## 4. Configurando o Git no computador e conectando ao repositório

Se ainda não tiver o Git instalado, baixe em [git-scm.com](https://git-scm.com) e instale. Depois, abra o terminal dentro da pasta [meu-portfolio](#) e rode:

```
git init                                # inicia um repositório Git local nesta pasta
git add .                               # adiciona todos os arquivos criados até agora
git commit -m "Estrutura inicial do projeto" # salva o primeiro commit com uma mensagem
git branch -M main                      # garante que a branch principal se chama "main"
git remote add origin https://github.com/SEU-USUARIO/meu-portfolio.git
                                         # conecta seu repositório local ao do GitHub
git push -u origin main                 # envia os arquivos para o GitHub pela primeira vez
```

**Dica:** troque `SEU-USUARIO` pelo seu nome de usuário real do GitHub. Se for pedido login, use seu usuário e, no lugar de senha, um *Personal Access Token* (o GitHub não aceita mais senha comum via terminal — veja como gerar um token na seção 9.1).

## 5. HTML da página principal — comentado linha a linha

Abra `index.html` e escreva o código abaixo. Cada linha está comentada explicando sua função:

```
<!DOCTYPE html>                                <!-- declara que é um documento HTML5 -->
<html lang="pt-br">                             <!-- define o idioma da página como português -->
<head>                                           <!-- início dos metadados da página -->
  <meta charset="UTF-8">                         <!-- garante acentuação correta (á, ç, ã...) -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
                                                <!-- faz o site se adaptar a celulares -->
  <title>Seu Nome – Portfólio</title>           <!-- título que aparece na aba do navegador -->
  <link rel="stylesheet" href="css/style.css">    <!-- importa o arquivo de estilos -->
</head>
<body>                                           <!-- início do conteúdo visível da página -->

  <header class="header">                       <!-- topo do site -->
    <h1>Seu Nome</h1>                          <!-- seu nome em destaque -->
    <p class="subtitle">Desenvolvedor(a) de Software</p>
                                                <!-- subtítulo/profissão -->
    <nav>                                         <!-- menu de navegação -->
      <a href="#sobre">Sobre</a>                 <!-- link interno para a seção Sobre -->
      <a href="#projetos">Projetos</a>           <!-- link interno para a seção Projetos -->
      <a href="#contato">Contato</a>            <!-- link interno para a seção Contato -->
    </nav>
  </header>

  <section id="sobre" class="secao">             <!-- seção "Sobre mim" -->
    <h2>Sobre mim</h2>
    <p>Escreva aqui um resumo curto sobre você.</p>
  </section>

  <section id="projetos" class="secao">          <!-- seção "Projetos" -->
    <h2>Projetos</h2>
    <div id="lista-projetos" class="grid-projetos"></div>
                                                <!-- div vazia: o JS insere os cards aqui -->
  </section>

  <section id="contato" class="secao">           <!-- seção "Contato" -->
    <h2>Contato</h2>
    <p>Email: seuemail@exemplo.com</p>
    <p>
      <a href="https://github.com/seu-usuario" target="_blank">GitHub</a> |
      <a href="https://linkedin.com/in/seu-usuario" target="_blank">LinkedIn</a>
    </p>
                                                <!-- target="_blank" abre em nova aba -->
  </section>

  <footer class="footer">                       <!-- rodapé -->
    <p>© 2026 Seu Nome. Todos os direitos reservados.</p>
  </footer>

  <script src="js/projetos.js"></script>        <!-- carrega os DADOS dos projetos primeiro -->
  <script src="js/main.js"></script>           <!-- depois carrega o script que EXIBE os projetos -->
</body>
</html>
```

## 6. CSS do tema escuro — comentado bloco a bloco

Crie `css/style.css`. Usaremos **variáveis CSS** ( `:root` ) para deixar fácil trocar as cores no futuro.

```
:root {
  --preto: #0d0d0d;          /* cor de fundo principal */
  --preto-claro: #1a1a1a;    /* fundo de cards e cabeçalho */
  --azul: #1e6fd9;           /* cor de destaque principal (links, títulos) */
  --azul-escuro: #123a73;    /* usado em bordas do admin */
  --vermelho: #d92b2b;       /* cor de destaque de atenção (hover, botões) */
  --texto: #e6e6e6;          /* cor do texto principal */
  --texto-suave: #a0a0a0;    /* cor de texto secundário */
}

* {
  margin: 0;                  /* remove margens padrão do navegador */
  padding: 0;                 /* remove espaçamentos internos padrão */
  box-sizing: border-box;     /* padding/borda não somam à largura total */
}

body {
  background-color: var(--preto); /* aplica o fundo preto em todo o site */
  color: var(--texto);           /* cor padrão do texto */
  font-family: 'Segoe UI', Arial, sans-serif;
  line-height: 1.6;             /* espaçamento entre linhas de texto */
}

.header {
  background-color: var(--preto-claro); /* fundo um pouco mais claro que o body */
  padding: 40px 20px;             /* espaço interno do cabeçalho */
  text-align: center;             /* centraliza o conteúdo */
  border-bottom: 3px solid var(--vermelho); /* linha vermelha na base do header */
}

.header h1 { color: var(--azul); font-size: 2.5rem; } /* nome em azul e grande */
.subtitle { color: var(--texto-suave); margin-top: 8px; } /* subtítulo em tom mais apagado */

nav { margin-top: 20px; }
nav a {
  color: var(--texto);
  text-decoration: none;         /* remove sublinhado padrão do link */
  margin: 0 15px;               /* espaçamento entre os links do menu */
  font-weight: bold;
  transition: color 0.3s;       /* anima a troca de cor no hover */
}
nav a:hover { color: var(--vermelho); } /* ao passar o mouse, link fica vermelho */

.secao {
  max-width: 900px;             /* largura máxima do conteúdo */
  margin: 60px auto;            /* centraliza a seção e dá espaço vertical */
  padding: 0 20px;              /* espaço lateral em telas pequenas */
}
.secao h2 {
  color: var(--azul);
  border-left: 4px solid var(--vermelho); /* barrinha vermelha ao lado do título */
  padding-left: 12px;
  margin-bottom: 20px;
}

.grid-projetos {
  display: grid;                 /* organiza os cards em grade */
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
                                  /* colunas responsivas automáticas */
  gap: 20px;                    /* espaço entre os cards */
}

.card-projeto {
  background-color: var(--preto-claro);
  border: 1px solid #2a2a2a;
  border-radius: 10px;          /* cantos arredondados */
  padding: 20px;
  transition: transform 0.25s, border-color 0.25s; /* anima o hover */
}
.card-projeto:hover {
  transform: translateY(-5px);   /* "levanta" o card ao passar o mouse */
  border-color: var(--azul);     /* borda fica azul no hover */
}
.card-projeto h3 { color: var(--vermelho); margin-bottom: 10px; }
.card-projeto p { color: var(--texto-suave); margin-bottom: 15px; font-size: 0.95rem; }
.card-projeto a {
  display: inline-block;
  background-color: var(--azul); /* botão azul de "Ver no GitHub" */
  color: #fff;
  padding: 8px 14px;
  border-radius: 6px;
  text-decoration: none;
  font-size: 0.9rem;
  transition: background-color 0.3s;
}
.card-projeto a:hover { background-color: var(--vermelho); } /* botão vira vermelho no hover */

.footer {
  text-align: center;
}
```



```
padding: 20px;
color: var(--texto-suave);
border-top: 1px solid #2a2a2a;
}

/* ===== Estilos exclusivos do painel ADM ===== */
.admin-container {
  max-width: 600px;
  margin: 60px auto;
  padding: 30px;
  background-color: var(--preto-claro);
  border-radius: 10px;
  border: 1px solid var(--azul-escuro);
}

.admin-container h2 { color: var(--azul); margin-bottom: 20px; }
.admin-container input,
.admin-container textarea {
  width: 100%;
  padding: 10px;
  margin-bottom: 15px;
  background-color: var(--preto);
  border: 1px solid #333;
  border-radius: 6px;
  color: var(--texto);
}

.admin-container button {
  background-color: var(--vermelho); /* botão de ação principal em vermelho */
  color: #fff;
  border: none;
  padding: 10px 20px;
  border-radius: 6px;
  cursor: pointer;
  font-weight: bold;
  transition: background-color 0.3s;
}

.admin-container button:hover { background-color: var(--azul); }

.item-lista-admin {
  display: flex;
  justify-content: space-between; /* nome de um lado, botão do outro */
  align-items: center;
  background-color: var(--preto);
  padding: 10px;
  border-radius: 6px;
  margin-bottom: 10px;
}

.item-lista-admin button { padding: 5px 10px; font-size: 0.8rem; }
```

## 7. JavaScript da página pública — comentado linha a linha

### 7.1 js/projetos.js

Este arquivo guarda os dados iniciais dos seus projetos:

```
// Lista de projetos exibidos no site.
// "let" (e não "const") porque essa lista pode ser sobrescrita pelo localStorage.
let projetos = [
  {
    titulo: "Sistema de Gestão RH",           // nome do projeto
    descricao: "Sistema em Django para RH.", // descrição curta
    link: "https://github.com/seu-usuario/gestao_rh" // link do repositório
  },
  {
    titulo: "Gerenciador de Academia",
    descricao: "Sistema em Java usando o padrão Facade.",
    link: "https://github.com/seu-usuario/gym-manager"
  }
]; // fim do array de projetos
```

### 7.2 js/main.js

Este script pega a lista de projetos e desenha os cards na tela:

```
function carregarProjetos() { // função que monta os cards na página
  const container = document.getElementById("lista-projetos");
  // pega a <div> vazia do HTML
  container.innerHTML = ""; // limpa qualquer conteúdo anterior

  const salvos = localStorage.getItem("projetos"); // verifica se há dados salvos no navegador
  const listaFinal = salvos ? JSON.parse(salvos) : projetos; // usa o salvo se existir, senão usa o padrão

  listaFinal.forEach(projeto => { // percorre cada projeto da lista
    const card = document.createElement("div"); // cria uma nova <div>
    card.className = "card-projeto"; // aplica a classe de estilo do card
    card.innerHTML = `
      <h3>${projeto.titulo}</h3> // título do projeto
      <p>${projeto.descricao}</p> // descrição do projeto
      <a href="${projeto.link}" target="_blank">Ver no GitHub</a>
    `; // botão de link para o repositório
    container.appendChild(card); // insere o card na página
  });
}

document.addEventListener("DOMContentLoaded", carregarProjetos);
// executa a função assim que a página carregar
```

## 8. Painel ADM — Opção simples (localStorage)

**Como funciona:** os projetos cadastrados ficam salvos apenas no navegador em que você usou o admin. Ótimo para organizar e testar; para publicar de verdade, você copia os dados finais para o `projetos.js` antes do `git push` (explicado ao final desta seção).

### 8.1 `admin.html`

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Painel ADM</title>
  <link rel="stylesheet" href="css/style.css">      <!-- reaproveita o mesmo tema escuro -->
</head>
<body>
  <div class="admin-container">                    <!-- caixa central do formulário -->
    <h2>Adicionar novo projeto</h2>
    <input type="text" id="input-titulo" placeholder="Título do projeto">
      <!-- campo do título -->
    <textarea id="input-descricao" placeholder="Descrição" rows="3"></textarea>
      <!-- campo da descrição -->
    <input type="text" id="input-link" placeholder="Link do GitHub">
      <!-- campo do link -->
    <button onclick="adicionarProjeto()">Adicionar</button>
      <!-- chama a função ao clicar -->

    <h2 style="margin-top:30px;">Projetos cadastrados</h2>
    <div id="lista-admin"></div>                  <!-- lista dos projetos já salvos -->
  </div>

  <script src="js/projetos.js"></script>
  <script src="js/admin.js"></script>
</body>
</html>
```

### 8.2 `js/admin.js`

```

function pegarListaAtual() { // retorna a lista mais atual de projetos
  const salvos = localStorage.getItem("projetos"); // busca dados salvos no navegador
  return salvos ? JSON.parse(salvos) : projetos; // usa salvos, senão usa o array padrão
}

function salvarLista(lista) { // grava a lista no localStorage
  localStorage.setItem("projetos", JSON.stringify(lista));
  // converte o array em texto (JSON) e salva
}

function adicionarProjeto() { // chamada pelo botão "Adicionar"
  const titulo = document.getElementById("input-titulo").value.trim();
  // lê o campo título, sem espaços extras
  const descricao = document.getElementById("input-descricao").value.trim();
  const link = document.getElementById("input-link").value.trim();

  if (!titulo || !link) { // validação simples
    alert("Preencha ao menos o título e o link.");
    return; // interrompe a função se faltar dado
  }

  const lista = pegarListaAtual(); // pega a lista existente
  lista.push({ titulo, descricao, link }); // adiciona o novo projeto no final
  salvarLista(lista); // salva a lista atualizada

  document.getElementById("input-titulo").value = ""; // limpa os campos do formulário
  document.getElementById("input-descricao").value = "";
  document.getElementById("input-link").value = "";

  renderizarListaAdmin(); // atualiza a lista exibida na tela
}

function removerProjeto(indice) { // remove um projeto pelo índice na lista
  const lista = pegarListaAtual();
  lista.splice(indice, 1); // remove 1 item na posição "indice"
  salvarLista(lista);
  renderizarListaAdmin();
}

function renderizarListaAdmin() { // desenha a lista de projetos no admin
  const container = document.getElementById("lista-admin");
  const lista = pegarListaAtual();
  container.innerHTML = ""; // limpa antes de redesenhar

  lista.forEach((projeto, indice) => { // "indice" identifica a posição do item
    const item = document.createElement("div");
    item.className = "item-lista-admin";
    item.innerHTML = `
      <span>${projeto.titulo}</span>
      <button onclick="removerProjeto(${indice})">Remover</button>
    `; // botão de remover já com o índice certo
    container.appendChild(item);
  });
}

document.addEventListener("DOMContentLoaded", renderizarListaAdmin);
// mostra a lista assim que a página abrir

```

**Publicando manualmente:** depois de cadastrar tudo no admin local, abra o Console do navegador (tecla F12 → aba "Console"), digite `localStorage.getItem("projetos")` e copie o resultado. Cole esse conteúdo dentro do array `projetos` em `js/projetos.js`, salve, e siga para a seção 10 para publicar.

## 9. Painel ADM — Opção avançada (API do GitHub)

Nesta opção, o painel escreve direto num arquivo `projetos.json` dentro do seu repositório, então qualquer visitante já vê os projetos novos, sem você precisar mexer em código.

### 9.1 Criar o Token de Acesso Pessoal

- 1 No GitHub, clique na sua foto (canto superior direito) → **Settings**.
- 2 No menu lateral esquerdo, role até **Developer settings**.
- 3 Clique em **Personal access tokens** → **Fine-grained tokens**.
- 4 Clique em **Generate new token**.
- 5 Dê um nome (ex: "token-admin-portfolio") e defina uma validade, por exemplo 90 dias.
- 6 Em **Repository access**, escolha **Only select repositories** e selecione apenas o repositório `meu-portfolio`.
- 7 Em **Permissions** → **Repository permissions**, encontre **Contents** e defina como **Read and write**.
- 8 Clique em **Generate token** e **copie o token exibido** (ele só aparece uma vez).

**Atenção:** esse token dá poder de escrita no seu repositório. Nunca o cole dentro de um arquivo `.js` que será publicado. No painel abaixo, ele é digitado manualmente pelo admin a cada uso e fica só na memória do navegador.

### 9.2 Criar o arquivo `projetos.json` no repositório

Na raiz do repositório (pelo site do GitHub, botão "Add file" → "Create new file", ou localmente), crie `projetos.json`:

```
[
  {
    "titulo": "Sistema de Gestão RH",           // nome do projeto
    "descricao": "Sistema em Django para RH.", // descrição
    "link": "https://github.com/seu-usuario/gestao_rh" // link do repositório
  }
]
```

### 9.3 Ajustar `js/main.js` para ler do JSON

```
async function carregarProjetos() {
  // "async" permite usar "await" dentro
  const container = document.getElementById("lista-projetos");
  container.innerHTML = "";

  const resposta = await fetch("projetos.json"); // busca o arquivo JSON no servidor
  const listaFinal = await resposta.json();      // converte a resposta em array JS

  listaFinal.forEach(projeto => {
    const card = document.createElement("div");
    card.className = "card-projeto";
    card.innerHTML = `
      <h3>${projeto.titulo}</h3>
      <p>${projeto.descricao}</p>
      <a href="${projeto.link}" target="_blank">Ver no GitHub</a>
    `;
    container.appendChild(card);
  });
}

document.addEventListener("DOMContentLoaded", carregarProjetos);
```

### 9.4 Campos extras em `admin.html`

```
<input type="text" id="input-usuario" placeholder="Seu usuário GitHub">
<!-- ex: joaosilva -->
<input type="text" id="input-repo" placeholder="Nome do repositório">
<!-- ex: meu-portfolio -->
<input type="password" id="input-token" placeholder="Token de acesso pessoal">
<!-- tipo "password" esconde os caracteres -->
```

### 9.5 Novo `js/admin.js` com a API do GitHub

```

async function pegarArquivoAtual() { // busca o projetos.json direto do GitHub
  const usuario = document.getElementById("input-usuario").value.trim();
  const repo = document.getElementById("input-repo").value.trim();
  const token = document.getElementById("input-token").value.trim();

  const url = `https://api.github.com/repos/${usuario}/${repo}/contents/projetos.json`;
  // endpoint da API para esse arquivo

  const resposta = await fetch(url, {
    headers: { Authorization: `token ${token}` } // autentica a requisição com o token
  });

  const dados = await resposta.json(); // dados vêm com conteúdo em base64
  const conteudoDecodificado = atob(dados.content); // decodifica de base64 para texto
  return {
    lista: JSON.parse(conteudoDecodificado), // transforma o texto em array JS
    sha: dados.sha, // identificador da versão do arquivo
    url
  };
}

async function salvarArquivo(lista, sha, url, token) { // escreve o arquivo atualizado no GitHub
  const novoConteudo = btoa(unescape(encodeURIComponent(JSON.stringify(lista, null, 2))));
  // converte o array de volta para base64

  await fetch(url, {
    method: "PUT", // PUT = criar/atualizar o arquivo
    headers: {
      Authorization: `token ${token}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      message: "Atualiza lista de projetos via painel ADM",
      // mensagem do commit gerado
      content: novoConteudo, // novo conteúdo do arquivo
      sha: sha // precisa informar a versão anterior
    })
  });
}

async function adicionarProjeto() { // função chamada pelo botão "Adicionar"
  const titulo = document.getElementById("input-titulo").value.trim();
  const descricao = document.getElementById("input-descricao").value.trim();
  const link = document.getElementById("input-link").value.trim();
  const token = document.getElementById("input-token").value.trim();

  if (!titulo || !link) {
    alert("Preencha ao menos o título e o link.");
    return;
  }

  const { lista, sha, url } = await pegarArquivoAtual(); // pega a lista e a versão atual
  lista.push({ titulo, descricao, link }); // adiciona o novo projeto
  await salvarArquivo(lista, sha, url, token); // envia a atualização ao GitHub

  alert("Projeto adicionado com sucesso!");
  document.getElementById("input-titulo").value = "";
  document.getElementById("input-descricao").value = "";
  document.getElementById("input-link").value = "";
}

```

Assim que o commit é feito pela API, o GitHub Pages detecta a mudança e atualiza o site publicado automaticamente em poucos segundos ou minutos.

## 10. Publicando as alterações no GitHub

Toda vez que você terminar de editar arquivos localmente (HTML, CSS, JS), envie as mudanças para o repositório com:

```
git add .                # seleciona todos os arquivos alterados
git commit -m "Atualiza estilos e projetos" # cria um commit descrevendo a mudança
git push                 # envia o commit para o GitHub
```

**Fluxo resumido:** editar arquivos → `git add .` → `git commit -m "mensagem"` → `git push` . Repita esse ciclo sempre que quiser publicar algo novo.

## 11. Hospedagem: GitHub Pages, Netlify e Vercel

### 11.1 GitHub Pages (recomendado, gratuito e mais simples)

- 1 No repositório do GitHub, clique na aba **Settings**.
- 2 No menu lateral, clique em **Pages**.
- 3 Em **Source**, selecione a branch **main** e a pasta / (**root**).
- 4 Clique em **Save**.
- 5 Aguarde alguns minutos. O GitHub mostrará o link final, no formato:

```
https://SEU-USUARIO.github.io/meu-portfolio/
```

**Importante para a Opção Avançada (seção 9):** o repositório precisa ser **público** para que o `fetch("projetos.json")` funcione sem autenticação no site publicado. O token só é usado dentro do `admin.html`, nunca na página pública.

### 11.2 Netlify (alternativa)

- 1 Crie uma conta em `netlify.com`.
- 2 Clique em **Add new site** → **Import an existing project**.
- 3 Conecte sua conta do GitHub e selecione o repositório `meu-portfolio`.
- 4 Deixe as configurações padrão (não há comando de build, pois o projeto é HTML/CSS/JS puro).
- 5 Clique em **Deploy site**.

### 11.3 Vercel (alternativa)

- 1 Crie uma conta em `vercel.com`.
- 2 Clique em **Add New** → **Project**.
- 3 Importe o repositório `meu-portfolio` do GitHub.
- 4 Não é necessário configurar build; clique em **Deploy**.

Tanto o Netlify quanto o Vercel atualizam o site automaticamente a cada `git push`, assim como o GitHub Pages.



## 12. Checklist final

| Etapa                                                          | Feito?                   |
|----------------------------------------------------------------|--------------------------|
| Estrutura de pastas criada                                     | <input type="checkbox"/> |
| Repositório criado no GitHub                                   | <input type="checkbox"/> |
| Git configurado e primeiro push realizado                      | <input type="checkbox"/> |
| <code>index.html</code> com header, sobre, projetos e contato  | <input type="checkbox"/> |
| <code>style.css</code> com tema escuro (preto, azul, vermelho) | <input type="checkbox"/> |
| <code>projetos.js</code> com a lista inicial                   | <input type="checkbox"/> |
| <code>main.js</code> exibindo os cards dinamicamente           | <input type="checkbox"/> |
| <code>admin.html</code> funcional (Opção A ou B escolhida)     | <input type="checkbox"/> |
| Testado localmente no navegador                                | <input type="checkbox"/> |
| GitHub Pages (ou Netlify/Vercel) configurado                   | <input type="checkbox"/> |
| Link final testado em outro dispositivo/aba anônima            | <input type="checkbox"/> |

Documentação gerada para orientar a criação de um portfólio pessoal estático com painel administrativo. Próximos passos sugeridos (opcionais): edição de projetos existentes no ADM, filtro por tecnologia, e login simples de usuário/senha para proteger o acesso ao `admin.html`.